Problem 1.2.23

ee25btech11023-Venkata Sai

March 26, 2026

1. Problem
2. Solution
   - Sampling
   - The Nyquist-Shannon Sampling Theorem
   - Aliasing
   - Calculation
   - Analysis on Undersampling
3. Analysis Plot
4. Python Code

Problem

The minimum frequency at which a signal comprising of 30 Hz, 50 Hz and 70 Hz should be sampled to avoid aliasing is . . . Hz.

Sampling

Sampling is the process of converting a continuous-time analog signal into a discrete-time digital signal. This is done by measuring the amplitude of the signal at uniform intervals, defined by the Sampling Frequency.

The Nyquist-Shannon Sampling Theorem

To ensure a signal can be perfectly reconstructed without losing information, it must be sampled at a rate at least twice the highest frequency component present in the signal. This minimum required rate is known as the Nyquist Rate.

$$f_s \geq 2 \times f_{max} \tag{1.1}$$

Aliasing

If the sampling frequency is lower than the Nyquist Rate, aliasing occurs. This is a phenomenon where high-frequency components appear as lower-frequency "aliases" in the reconstructed data. These fake lower frequencies distort the original signal and cannot be removed once sampled.

Calculation

Given frequencies $= \{30, 50, 70\}$ Hz

$$f_{max} = 70 \text{ Hz} \tag{1.2}$$

From Nyquist-Shannon Theorem,

$$f_s = 2 \times f_{max} \tag{1.3}$$

$$f_s = 2 \times 70 \text{ Hz} \implies f_s = 140 \text{ Hz} \tag{1.4}$$

Analysis on Undersampling

If this signal were sampled at only 100 Hz (which is $\leq$ 140 Hz):

1. The 30 Hz and 50 Hz components would be captured correctly.
2. The 70 Hz component would alias. It would appear as a 30 Hz signal interfering with the original 30 Hz component and causing irreversible distortion.

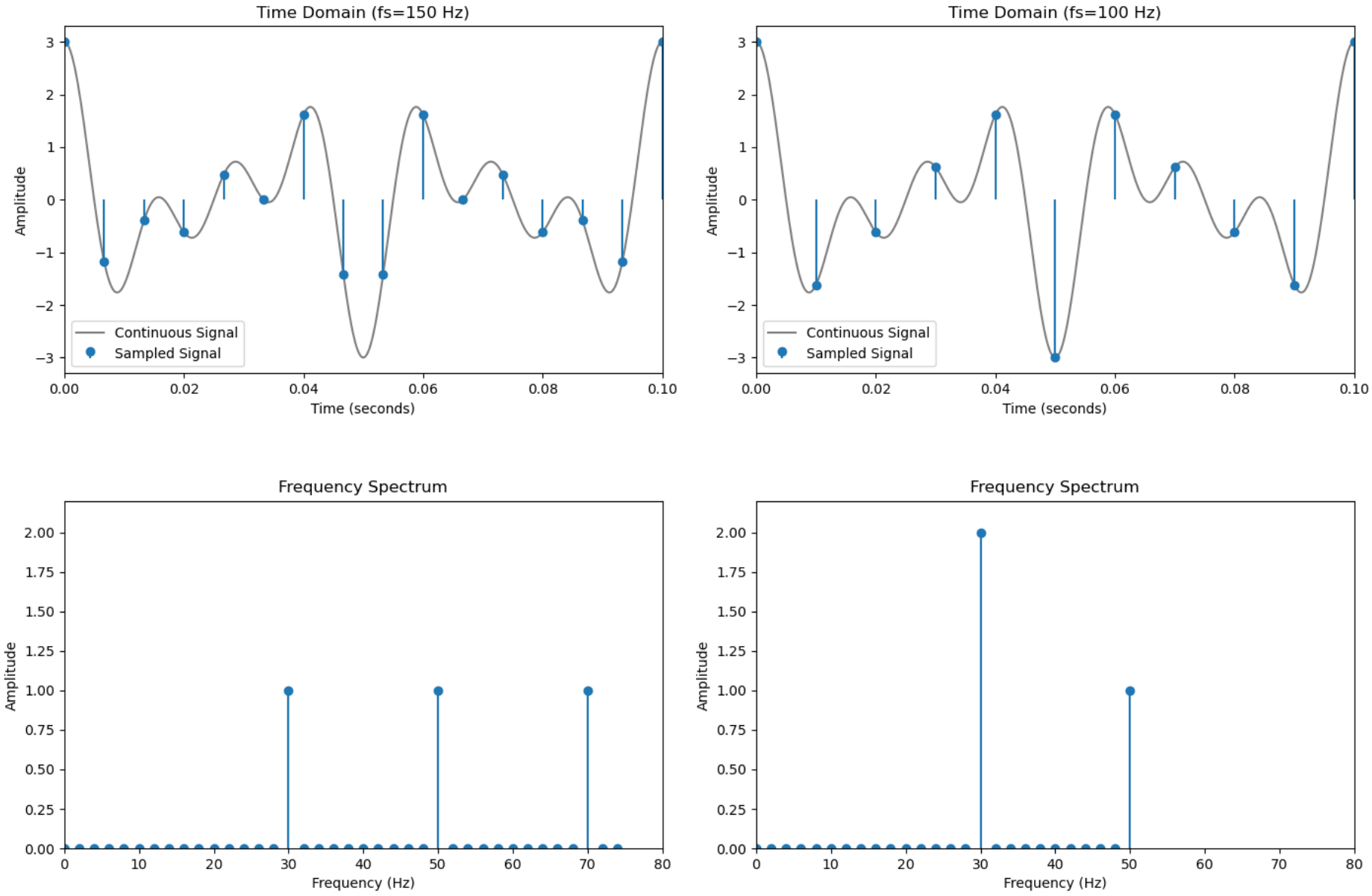
To find the alias frequency f_a for any signal f_{in} sampled at a rate f_s :

$$f_a = |f_s - f_{in}| \tag{1.5}$$

$$f_a = |100 - 70| = 30 \text{ Hz} \tag{1.6}$$

In this case, this 30 Hz might get overlapped with the original 30 Hz signal. This make impossible to seperate the aliased one and the original one later

Analysis Plot



Python Code for Plotting

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import rfft, rfftfreq

freqs = [30, 50, 70]

def sig(t):
    return sum(np.cos(2*np.pi*f*t) for f in freqs)

duration = 0.5
t_cont = np.linspace(0, duration, 2000, endpoint=False)
x_cont = sig(t_cont)

fig, ax = plt.subplots(2, 2, figsize=(12,6))

for i, fs in enumerate([150, 100]):
    t = np.arange(0, duration, 1/fs)
    x = sig(t)
```

Python Code for Plotting

```
  # FFT
    N = len(x)
    X = rfft(x)
    f = rfftfreq(N, 1/fs)
    amp = np.abs(X)/(N/2)
    amp[0] /= 2
    if N % 2 == 0:
        amp[-1] /= 2

# Time plot
    ax[0,i].plot(t_cont, x_cont, 'gray', label='Continuous Signal
        ')
    ax[0,i].stem(t, x, basefmt= , label='Sampled Signal')
    ax[0,i].set_xlim(0, 0.1)
    ax[0,i].set_title(fTime Domain (fs={fs} Hz))
    ax[0,i].set_xlabel(Time (seconds))
    ax[0,i].set_ylabel(Amplitude)
    ax[0,i].legend()
```

Python Code for Plotting

```
    # Frequency plot
    ax[1,i].stem(f, amp, basefmt= )
    ax[1,i].set_xlim(0, 80)
    ax[1,i].set_ylim(0, 2.2)
    ax[1,i].set_title(Frequency Spectrum)
    ax[1,i].set_xlabel(Frequency (Hz))
    ax[1,i].set_ylabel(Amplitude)

plt.tight_layout()
plt.show()
```